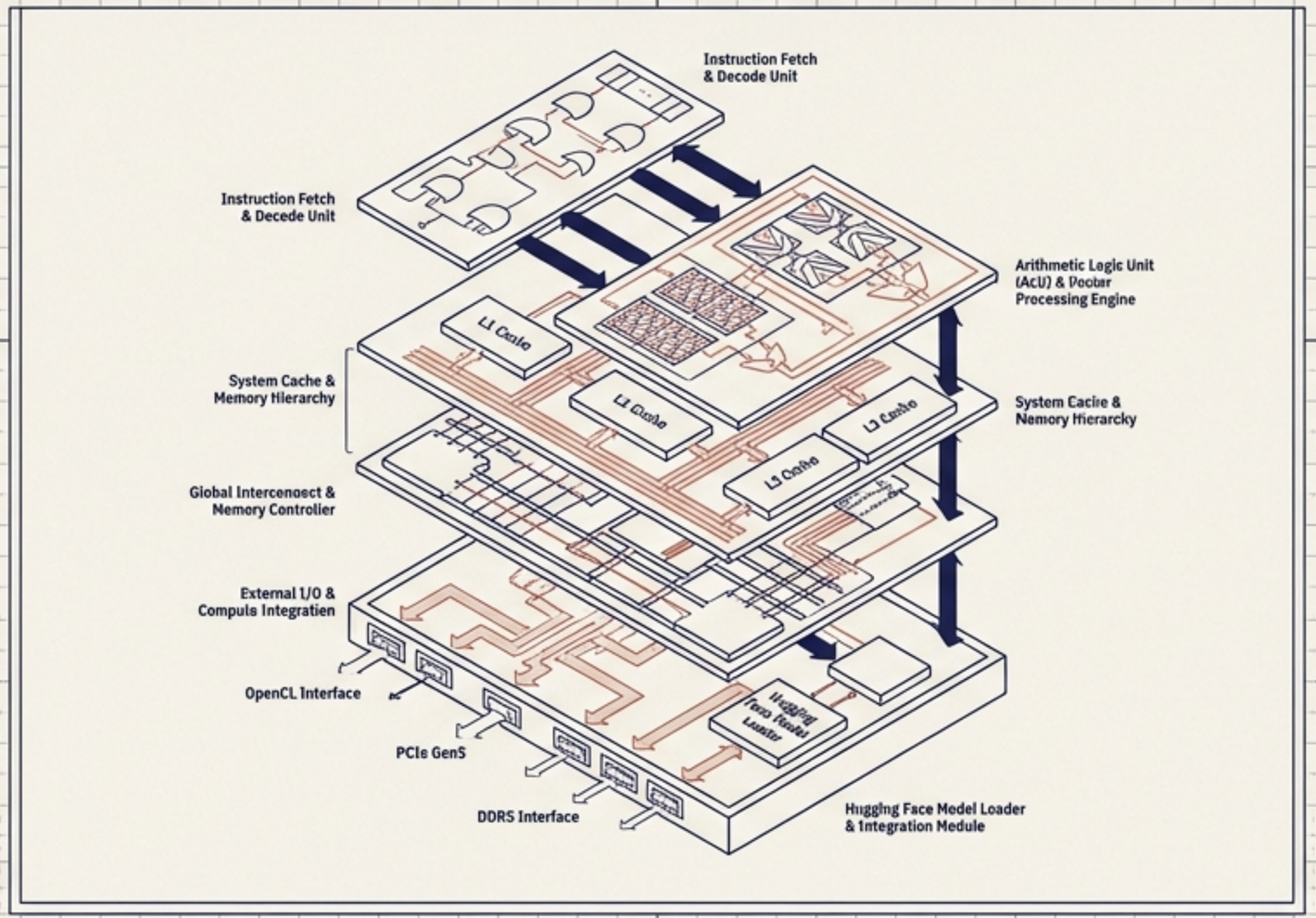
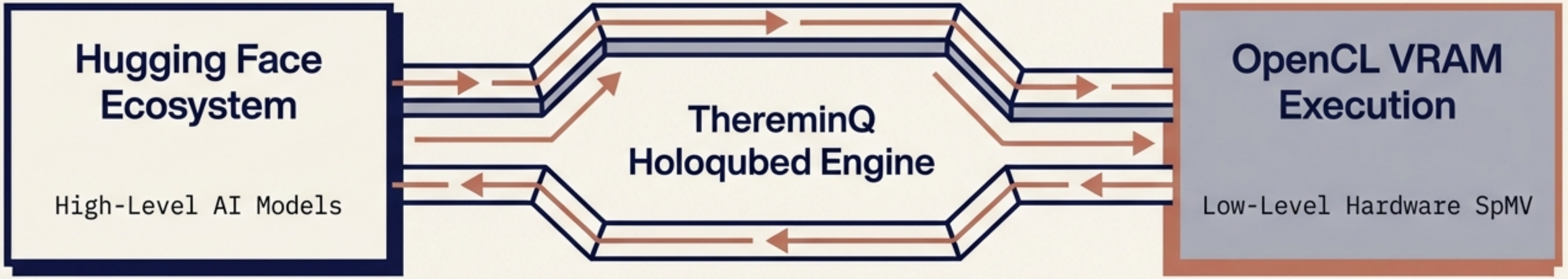
TheremiQ Holoqubed: Core Engine Architecture

System Blueprint and Directory Topography
Mapping OpenCL Execution & Hugging Face Integrations



NotebookLM

Bridging Hugging Face Models with Bare-Metal OpenCL Execution



The Problem

Standard inference engines abstract away direct hardware control, limiting custom VRAM VRAM optimizations.

The Solution

A specialized directory housing execution scripts, prototyping looms, and C++ extensions built for direct OpenCL Sparse Matrix-Vector Multiplication (SpMV).

The Output

A cohesive pipeline converting A cohesive pipeline converting high-level model structures into .holo dictionaries for optimized hardware ingestion.

NotebookLM

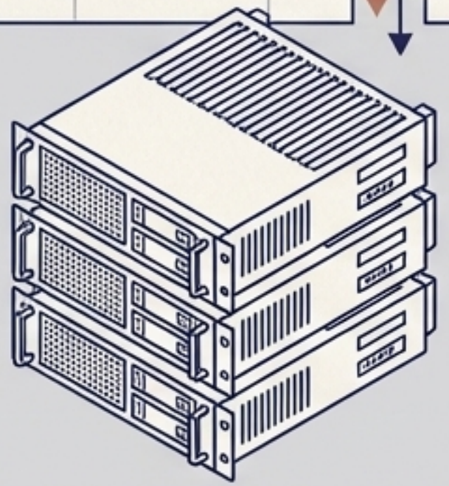
Mapping the Engine's Functional Topography



Module A: The Laboratory

`Directory: concept/`

Conceptual prototypes and verification harnesses for testing early integrations.

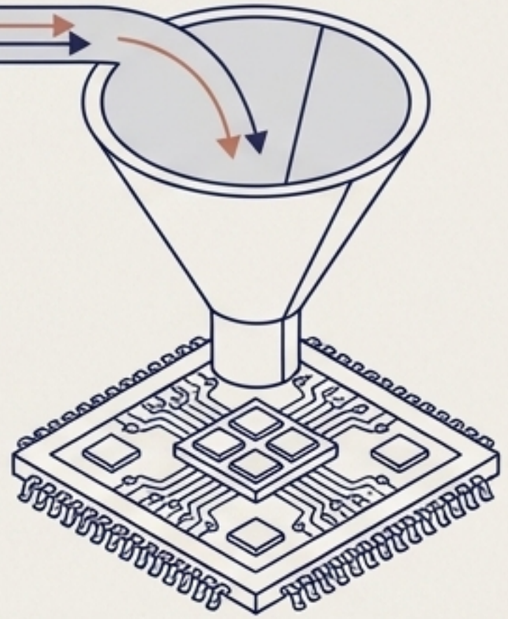


Module B: **The Production Core**

`Directory: python-c_hf-implementation/`

The primary Python/C++ implementation bridging Hugging Face converters with OpenCL extensions.

Module C: **The Simulator**

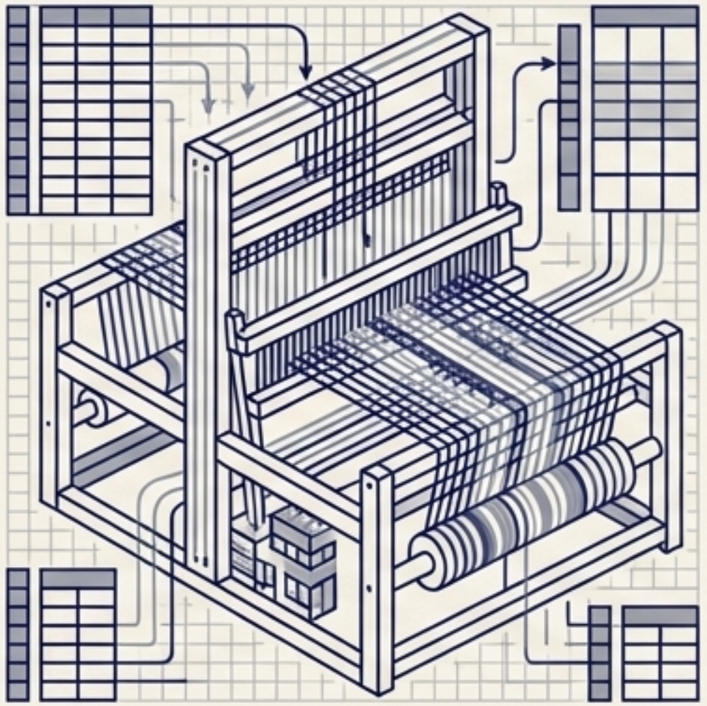


`File: holo_check.py`

The Ingress Loader and Simulator for verifying the SpMV execution pipeline.

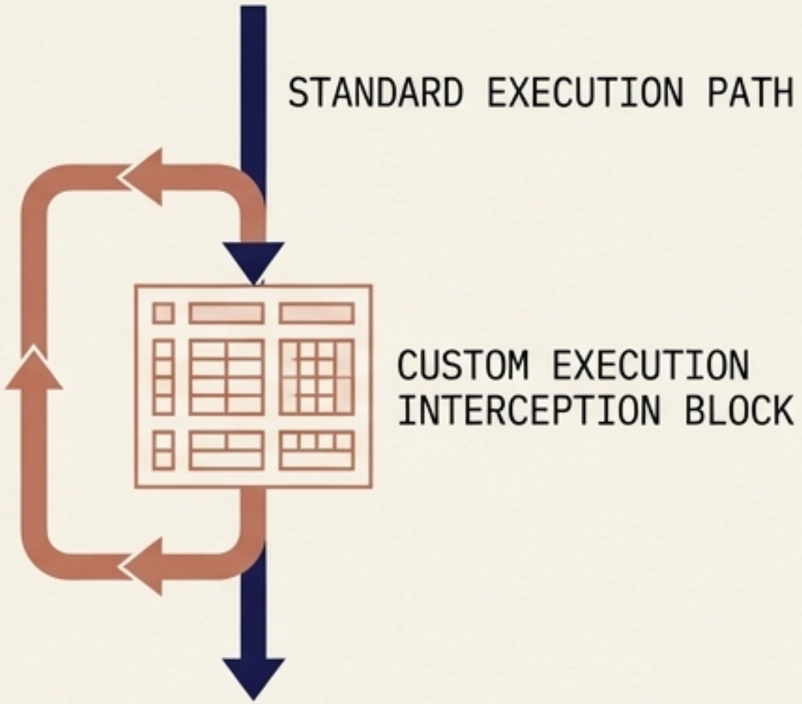
NotebookLM

Validating Logic via the Conceptual Harness



PyOpenCL Loom

Acts as the primary testing framework to weave standard Python commands directly into experimental OpenCL data structures. Provides early-stage verification before strict C++ compilation.

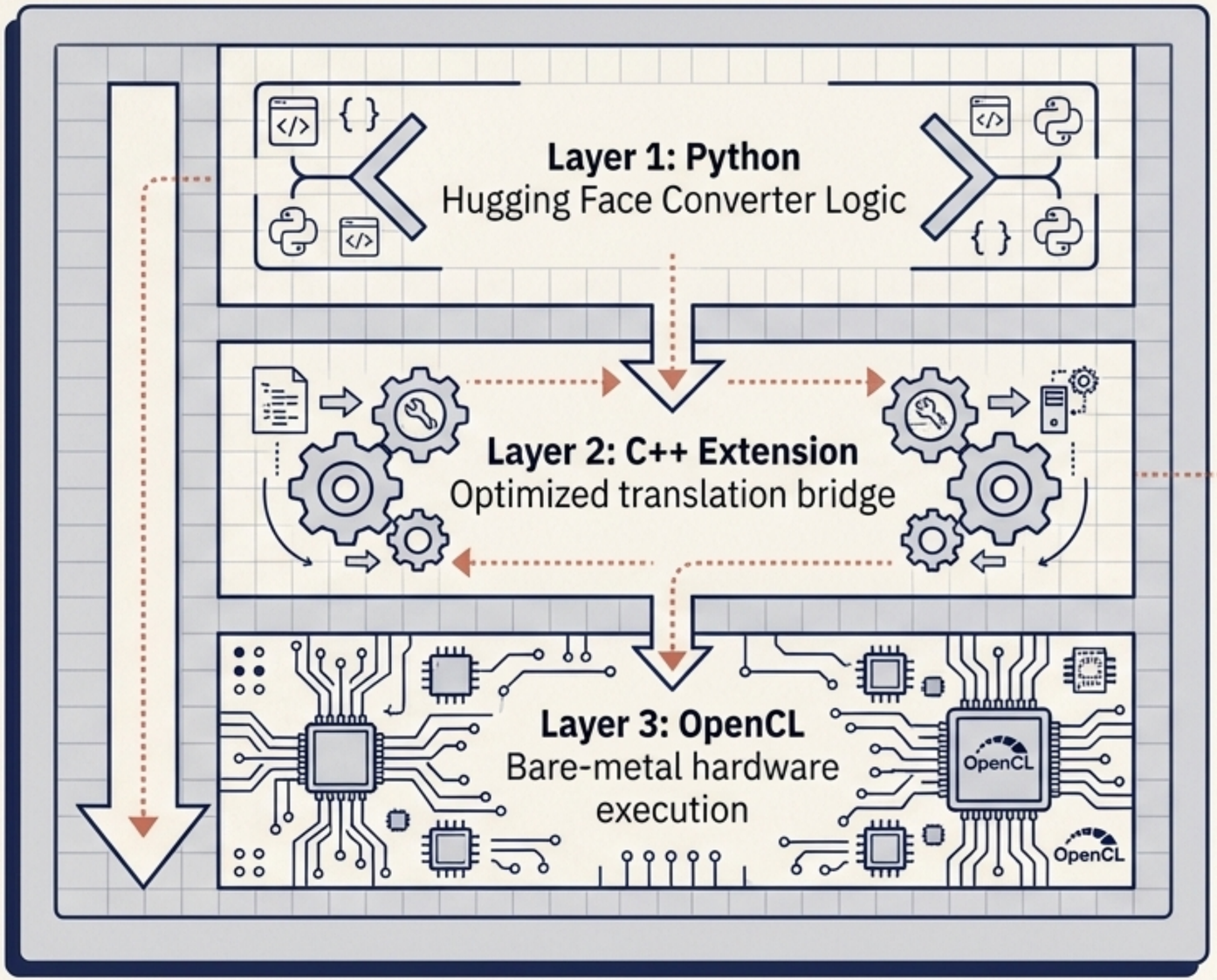


Hugging Face Monkey-Patching

A runtime interception technique used in the prototype phase. Forcibly injects custom hardware execution paths into standard HF models without permanently altering their source code.

NotebookLM

The Python-to-C++ Implementation Stack



Variant: Multithreaded Implementation

Path: `multithreaded-python-c_hf-implementation`

Utilizes concurrent execution pathways for exponentially higher throughput.

Variant: Hilbert Space Implementation

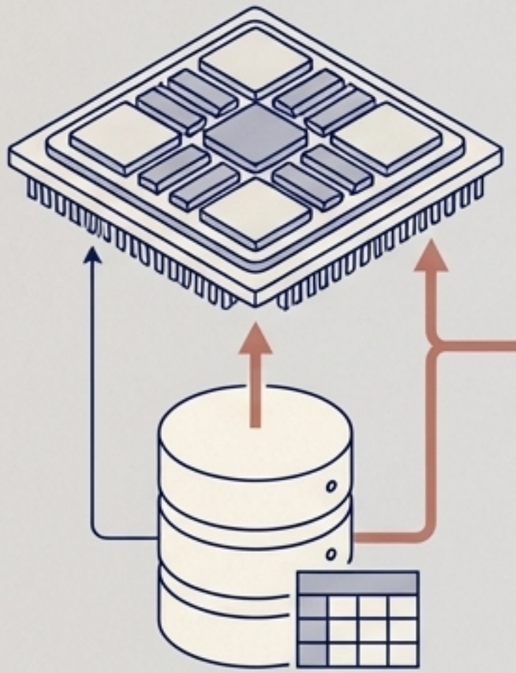
Path: `hilberspace-python-c_hf-implementation`

Experimental geometric spatial memory mapping for inference optimization.

NotebookLM

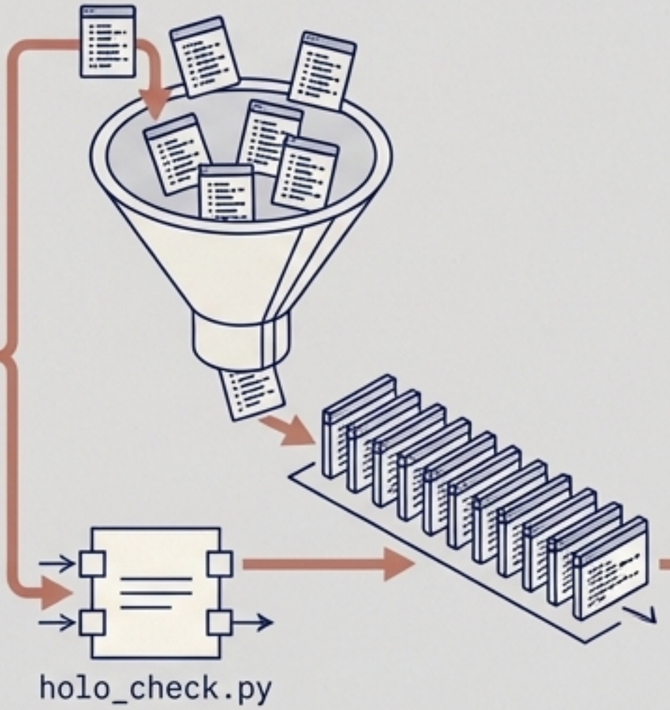
Simulating VRAM Execution Through the Ingress Pipeline

Step 1: CPU Query Planner



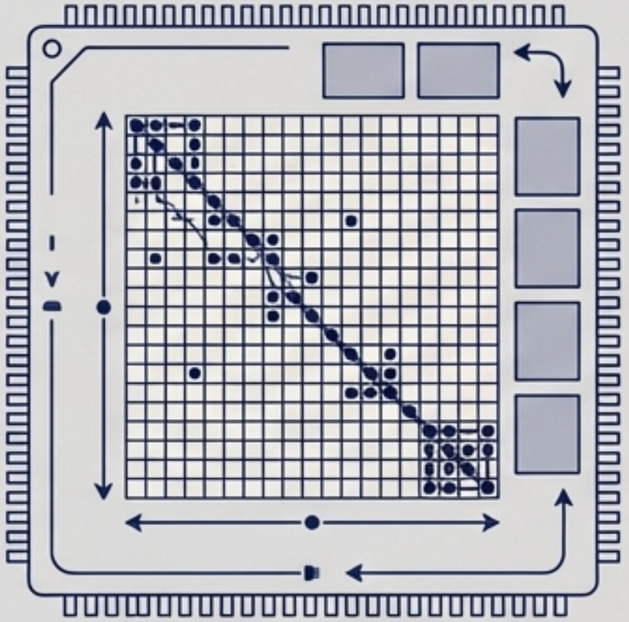
The planner accesses and loads neural network layers stored in a highly optimized `.holo` dictionary.

Step 2: The Ingress Loader



Located in `holo_check.py`, this mechanism simulates the staging and transfer of loaded `.holo` structures, prepping them for hardware ingestion.

Step 3: Direct VRAM Execution



Executes OpenCL Sparse Matrix-Vector Multiplication (SpMV) directly in VRAM to verify the integrity of the full pipeline.

NotebookLM

Aligning Prototyping and Production Capabilities

	Target Module: `concept/`	Target Module: `python-c_hf-implementation/`
Primary Language	Python (PyOpenCL)	Hybrid (Python / C++ OpenCL)
Core Purpose	Architecture Verification & Testing	High-Performance Model Integration
HF Integration Strategy	Runtime Monkey-Patching	Dedicated C++ Model Converter
Execution Focus	Logic Validation via Loom	Direct VRAM SpMV Optimization

Synthesis: The concept layer ensures rapid verification of non-standard integrations, while the implementation stack locks in that logic into a highly optimized, bare-metal OpenCL pipeline.

NotebookLM